

# Reusable Dense Linear Solving over $\mathbb{F}_2$ for Fixed-Matrix Many-RHS Systems: From Bit-Packed Gaussian Elimination to Block and M4RI-Style Acceleration

Theodore Magnus Øen

## Abstract

We study exact linear solving over  $\mathbb{F}_2$  in the fixed-matrix, many-right-hand-side setting. Given a fixed matrix  $A$  and many right-hand sides  $b^{(1)}, \dots, b^{(Q)}$ , repeatedly eliminating the augmented matrix  $[A \mid b^{(i)}]$  wastes the matrix-dependent work. We formalize a preprocessing-query framework in which Gaussian elimination is performed once on  $A$ , while the induced row transformations, column permutation, rank, pivot structure, and echelon form are stored and reused.

In the basic bit-packed model, dense square preprocessing costs  $\Theta(n^3/w)$  word operations, where  $w$  is the machine word size, and each online right-hand side can be solved in  $\Theta(n^2)$  time. Therefore the total cost for  $Q$  right-hand sides changes from  $\Theta(Qn^3/w)$  for repeated elimination to  $\Theta(n^3/w + Qn^2)$ . We then refine the framework in two directions. First, if the right-hand sides are available in a batch, they can be stored as a matrix  $B \in \mathbb{F}_2^{m \times Q}$ , allowing the recorded row operations to be applied to  $w_R$  right-hand sides per word. Second, the preprocessing phase can be replaced by block Gaussian elimination or M4RI-style Four Russians elimination, reducing dense elimination cost by a logarithmic factor up to table-construction and cache effects.

Finally, we explain how the same reusable  $\mathbb{F}_2$  solver appears as a subroutine in iterative lifting modulo  $2^k$ , and how composite moduli are handled by CRT decomposition into prime-power systems. The paper does not claim novelty for Gaussian elimination, the Four Russians method, Hensel-style lifting, or the Chinese Remainder Theorem individually. Its purpose is to present a coherent implementation-oriented framework around the fixed-matrix many-RHS abstraction.

**Keywords.** fixed matrix; multiple right-hand sides; Gaussian elimination;  $\mathbb{F}_2$ ; bitset; M4RI; Four Russians; block elimination; lifting modulo  $2^k$ ; Chinese Remainder Theorem

## 1 Introduction

Solving linear systems over finite fields is a core primitive in computational algebra, coding theory, cryptanalysis, and algorithm design. In many applications the coefficient matrix is fixed, while the right-hand side varies across many queries. Examples include repeated feasibility checks under the same binary constraint system, syndrome-related computations in coding theory, linear subproblems inside cryptanalytic workflows, and correction equations arising in modular lifting.

The classical principle of eliminating or factoring a fixed matrix once and then solving for multiple right-hand sides is well known in numerical linear algebra and exact linear algebra[1, 2]. Over  $\mathbb{F}_2$ , this principle becomes especially attractive because addition and subtraction are both xor, and dense rows can be packed into machine words. High-performance libraries such as M4RI exploit this representation together with block methods and the Method of the Four Russians[10, 11, 12].

This paper organizes these ideas around a single abstraction: the fixed-matrix many-RHS solver. During preprocessing, the matrix-dependent structure of  $A$  is extracted once. During each query, only the right-hand side is transformed and solved against the preprocessed echelon form. This clean separation gives a direct complexity transition from repeated cubic elimination to one preprocessing phase plus quadratic queries.

The paper has three goals. First, it gives a formal and implementation-oriented description of reusable elimination over  $\mathbb{F}_2$ . Second, it explains how batch right-hand sides and M4RI-style block elimination strengthen the same framework. Third, it connects the reusable  $\mathbb{F}_2$  primitive to iterative lifting modulo  $2^k$  and CRT decomposition for general composite moduli.

## 2 Problem Formulation

**Definition 1** (Fixed-matrix many-RHS problem). *Let  $R$  be a ring or field. Given a fixed matrix*

$$A \in R^{m \times n}$$

*and right-hand sides*

$$b^{(1)}, b^{(2)}, \dots, b^{(Q)} \in R^m,$$

*the fixed-matrix many-RHS problem asks, for each  $i$ , to decide whether*

$$Ax = b^{(i)}$$

*is solvable over  $R$ , and if it is solvable, to construct at least one solution  $x^{(i)} \in R^n$ .*

In this paper the main coefficient domain is  $R = \mathbb{F}_2$ . Later sections discuss extensions to  $\mathbb{F}_p$ , to the residue ring  $\mathbb{Z}/2^k\mathbb{Z}$ , and to general composite moduli through CRT.

**Definition 2** (Preprocessing-query solver). *A preprocessing-query solver consists of two phases.*

- (i) *The preprocessing phase receives  $A$  and constructs a state  $S(A)$  depending only on  $A$ .*
- (ii) *The query phase receives  $S(A)$  and a right-hand side  $b$ , then decides solvability and constructs one solution if possible.*

*The purpose is to move all expensive matrix-dependent work into preprocessing.*

Over a field, if  $Ax = b$  is solvable and  $x_0$  is one particular solution, then the full solution set is

$$x_0 + \ker(A).$$

Thus all right-hand sides share the same homogeneous solution structure. Only the particular solution depends on  $b$ .

## 3 Reusable Elimination over $\mathbb{F}_2$

### 3.1 Preprocessed solver state

Let  $A \in \mathbb{F}_2^{m \times n}$ . Gaussian elimination over  $\mathbb{F}_2$  uses only row swaps and row xors. Column swaps may also be used to simplify pivot selection; they do not affect the right-hand side, but they permute the unknowns.

**Definition 3** (Preprocessed state over  $\mathbb{F}_2$ ). *A valid preprocessed state for  $A$  is a tuple*

$$S(A) = (U, \Omega, P, r, \text{piv}, \text{free}),$$

*where:*

- (i)  $\Omega = (\omega_1, \dots, \omega_t)$  is a sequence of elementary row operations;
- (ii)  $E = E_t E_{t-1} \dots E_1$  is the invertible matrix represented implicitly by  $\Omega$ ;
- (iii)  $P$  is the accumulated column permutation matrix;
- (iv)  $U = EAP$  is in echelon form or reduced echelon form;
- (v)  $r = \text{rank}(A)$ ;
- (vi) `piv` records pivot columns;
- (vii) `free` records non-pivot columns.

The matrix  $E$  is not stored explicitly in the basic implementation. It is represented by replaying  $\Omega$ .

### 3.2 Preprocessing algorithm

---

**Algorithm 1** Bit-packed preprocessing over  $\mathbb{F}_2$

---

**Require:**  $A \in \mathbb{F}_2^{m \times n}$

**Ensure:** A preprocessed state  $S(A)$

```

1:  $U \leftarrow A, \Omega \leftarrow \emptyset, P \leftarrow I_n, r \leftarrow 0$ 
2: for columns  $c = 0, 1, \dots, n - 1$  do
3:   if  $r = m$  then
4:     break
5:   end if
6:   Find a nonzero entry  $U_{i,j} = 1$  with  $i \geq r, j \geq c$ 
7:   if no such entry exists then
8:     break
9:   end if
10:  if  $j \neq c$  then
11:    Swap columns  $j$  and  $c$  in  $U$ , and update  $P$ 
12:  end if
13:  if  $i \neq r$  then
14:    Swap rows  $i$  and  $r$ , and append the swap to  $\Omega$ 
15:  end if
16:  Record  $c$  as the pivot column of row  $r$ 
17:  for rows  $i' = r + 1, r + 2, \dots, m - 1$  do
18:    if  $U_{i',c} = 1$  then
19:       $U_{i'} \leftarrow U_{i'} + U_r$ , and append this xor to  $\Omega$ 
20:    end if
21:  end for
22:   $r \leftarrow r + 1$ 
23: end for
24: Optionally eliminate above pivots to obtain reduced echelon form, recording the row xors
   in  $\Omega$ 
25: return  $S(A) = (U, \Omega, P, r, \text{piv}, \text{free})$ 

```

---

### 3.3 Online query algorithm

---

**Algorithm 2** Single-RHS query over  $\mathbb{F}_2$

---

**Require:** State  $S(A) = (U, \Omega, P, r, \text{piv}, \text{free})$ , RHS  $b \in \mathbb{F}_2^m$

**Ensure:** Either UNSOLVABLE or one solution  $x$

```

1: Compute  $b' = Eb$  by replaying all row operations in  $\Omega$  on  $b$ 
2: for each zero row index  $i \geq r$  do
3:   if  $b'_i = 1$  then
4:     return UNSOLVABLE
5:   end if
6: end for
7: Set all free variables of  $y$  to 0
8: for pivot rows from bottom to top do
9:   Compute the pivot variable by back substitution in  $Uy = b'$ 
10: end for
11: return  $x = Py$ 

```

---

**Theorem 1** (Correctness). *Let  $S(A) = (U, \Omega, P, r, \text{piv}, \text{free})$  be a valid preprocessed state satisfying  $U = EAP$ . For every  $b \in \mathbb{F}_2^m$ , Algorithm 2 returns UNSOLVABLE if and only if  $Ax = b$  has no solution. If it returns  $x$ , then  $Ax = b$ .*

*Proof.* The row-operation matrix  $E$  is invertible, and the column permutation matrix  $P$  is also invertible. For  $x = Py$ ,

$$Ax = b \iff APy = b \iff EAPy = Eb \iff Uy = Eb.$$

Therefore the original system is equivalent to the transformed echelon system. Since  $U$  has rank  $r$ , all rows below the pivot rows are zero rows. The transformed system is inconsistent exactly when some zero row has RHS equal to 1. If no such contradiction occurs, back substitution constructs  $y$  satisfying  $Uy = Eb$ , and hence  $x = Py$  satisfies  $Ax = b$ .  $\square$

**Theorem 2** (Solution-space representation). *If  $Ax = b$  is solvable and  $x_0$  is the particular solution returned by Algorithm 2, then*

$$\{x \in \mathbb{F}_2^m : Ax = b\} = x_0 + \ker(A).$$

*A basis of  $\ker(A)$  is obtained from the same echelon form by assigning one free variable to 1, all other free variables to 0, solving pivot variables by back substitution, and applying  $P$ .*

*Proof.* If  $x$  and  $x_0$  are both solutions, then  $A(x - x_0) = 0$ , so  $x - x_0 \in \ker(A)$ . Conversely, if  $v \in \ker(A)$ , then  $A(x_0 + v) = b$ . The kernel-basis construction is the standard free-variable construction for the homogeneous echelon system  $Uy = 0$ , followed by the inverse coordinate restoration induced by  $P$ .  $\square$

### 3.4 Basic bit-packed complexity

Let  $w$  be the machine word size and

$$W = \left\lceil \frac{n}{w} \right\rceil.$$

In dense bit-packed representation, each row xor costs  $O(W)$  word operations.

**Theorem 3** (Basic dense complexity). *Let  $A \in \mathbb{F}_2^{m \times n}$  have rank  $r$ . The bit-packed preprocessing algorithm costs*

$$O(rmW)$$

word operations. In the dense square case  $m = n$  and  $r = \Theta(n)$ , this is

$$O\left(\frac{n^3}{w}\right).$$

If  $M = |\Omega|$ , then one online RHS query costs

$$O(M + rn).$$

In the dense square case  $M = O(n^2)$ , so a single query costs  $O(n^2)$ . Thus  $Q$  online queries cost

$$O\left(\frac{n^3}{w} + Qn^2\right),$$

whereas repeated elimination costs

$$O\left(Q\frac{n^3}{w}\right).$$

**Remark 1.** The bound  $O(rn)$  for back substitution is conservative. If pivot rows are packed and dot products are implemented using word-level parity operations, this term can also benefit from word parallelism. The main asymptotic comparison, however, is already captured by the transition from repeated cubic elimination to one cubic preprocessing phase plus quadratic queries.

## 4 Batch Right-Hand Sides

The online algorithm treats each right-hand side independently. If all or many right-hand sides are available at once, they should be stored as a matrix

$$B = [b^{(1)} \ b^{(2)} \ \dots \ b^{(Q)}] \in \mathbb{F}_2^{m \times Q}.$$

Then the same row-operation script  $\Omega$  can be replayed on  $B$  instead of on each column separately.

**Definition 4** (Batch RHS representation). Let  $w_R$  be the machine word size used for packing RHS columns. Store each row of  $B$  as  $\lceil Q/w_R \rceil$  words. A row xor on  $B$  then processes  $w_R$  right-hand sides per word.

**Theorem 4** (Batch query cost). Assume the preprocessed state  $S(A)$  has row-operation sequence  $\Omega$  with  $|\Omega| = M$ . Applying all recorded row operations to the RHS batch  $B \in \mathbb{F}_2^{m \times Q}$  costs

$$O\left(M \left\lceil \frac{Q}{w_R} \right\rceil\right)$$

word operations. In the dense square case  $M = O(n^2)$ , this becomes

$$O\left(n^2 \left\lceil \frac{Q}{w_R} \right\rceil\right)$$

for the row-transformation part.

*Proof.* Each recorded row swap or row xor is applied to two rows of  $B$ . A row of  $B$  has  $\lceil Q/w_R \rceil$  packed words. Therefore each row operation costs  $O(\lceil Q/w_R \rceil)$  word operations, and replaying  $M$  operations gives the stated bound.  $\square$

After applying row operations to  $B$ , zero-row consistency can be checked for all right-hand sides in packed form. The back-substitution phase can also be applied in batch, producing a solution matrix

$$X = [x^{(1)} \ x^{(2)} \ \dots \ x^{(Q)}].$$

Thus the many-RHS setting should not always be viewed as  $Q$  independent online queries. When the RHS data are available in batches, the query side itself becomes a packed matrix computation.

## 5 Block and M4RI-Style Acceleration

### 5.1 Block Gaussian elimination

The basic bit-packed algorithm eliminates one pivot column at a time. A block algorithm groups several pivot columns and pivot rows together. This improves locality and reduces overhead by turning many small row operations into larger structured updates.

Block elimination does not change the algebraic invariant. The preprocessed state still represents

$$U = EAP,$$

where  $E$  is invertible and  $P$  is a permutation matrix. The difference is how the row transformations are computed and stored.

### 5.2 Method of the Four Russians over $\mathbb{F}_2$

M4RI-style elimination applies the Method of the Four Russians to dense matrices over  $\mathbb{F}_2$ . The basic idea is to process a block of  $k$  pivot rows at once. For these  $k$  rows, precompute all  $2^k$  possible linear combinations. For a target row, the bits in the  $k$  pivot columns form a mask. Instead of xoring up to  $k$  pivot rows into the target row, one xors a single precomputed table entry.

---

#### Algorithm 3 One M4RI-style elimination block

---

**Require:** Current matrix block with  $k$  pivot rows

**Ensure:** Target rows are cleared in the  $k$  pivot columns

- 1: Build a table  $T$  containing all  $2^k$  xor combinations of the  $k$  pivot rows
  - 2: **for** each target row  $R$  **do**
  - 3:     Read the  $k$  pivot-column bits of  $R$  as a mask  $\mu$
  - 4:      $R \leftarrow R + T[\mu]$
  - 5: **end for**
- 

With  $k \approx \log n$ , the dense elimination cost can be reduced by an additional logarithmic factor, up to table-construction and memory-hierarchy effects. A common high-level model is

$$O\left(\frac{n^3}{w \log n}\right)$$

for dense square elimination over  $\mathbb{F}_2$ , although actual performance depends on cache behavior, table layout, SIMD support, density, and implementation details[11, 12].

**Remark 2** (Role of M4RI in this paper). *M4RI is an acceleration of the preprocessing phase, not a change in the mathematical correctness argument. Once elimination has produced a valid state  $U = EAP$ , all query-side correctness proofs remain identical. Therefore M4RI can be treated as a drop-in replacement for the ordinary bit-packed elimination engine.*

### 5.3 Batch RHS with block transformations

For batch right-hand sides, the same block idea can be applied to the RHS matrix  $B$ . If a block of  $k$  pivot-row operations is represented by all  $2^k$  combinations, then rows of  $B$  can be updated by table lookup instead of by  $k$  separate row operations.

A conservative model for the full dense many-RHS framework is therefore:

$$\text{preprocessing: } O\left(\frac{n^3}{w \log n}\right),$$

$$\text{batch query: } O\left(n^2 \left\lceil \frac{Q}{w_R} \right\rceil\right)$$

with ordinary replay, and potentially another logarithmic improvement in the row-update count if block transformations are also used on the RHS batch. Since the exact constant depends strongly on implementation, this paper treats the logarithmic query improvement as an engineering optimization rather than a central theorem.

#### 5.4 Explicit transformation matrix viewpoint

There is another clean way to view the batch setting. If the preprocessing phase explicitly constructs the transformation matrix  $E$ , then for a batch  $B$  one directly computes

$$B' = EB.$$

This is a dense matrix multiplication over  $\mathbb{F}_2$ , precisely the operation for which M4RI-style algorithms are well suited. Similarly, once a canonical particular-solution map  $T$  is fixed, the solutions for a batch may be written as

$$X = TB.$$

This viewpoint can be faster for large batches, at the cost of storing additional dense transformation matrices.

**Remark 3** (Dense versus sparse regimes). *M4RI-style methods are designed for dense matrices over  $\mathbb{F}_2$ . If  $A$  is extremely large and sparse, sparse elimination, block Lanczos, or block Wiedemann-type algorithms may be more appropriate. The framework in this paper is primarily a dense or moderately dense exact linear algebra framework.*

## 6 Extension to General Finite Fields

The preprocessing-query abstraction extends to general finite fields  $\mathbb{F}_p$ , where  $p$  is prime. Standard background on finite fields and computational algebra can be found in [3, 4, 5].

**Theorem 5** (Extension to  $\mathbb{F}_p$ ). *Let  $A \in \mathbb{F}_p^{m \times n}$ . A preprocessing-query solver can be constructed by recording row swaps, row additions by scalar multiples, row scalings by nonzero scalars, column permutations, and pivot information. If preprocessing produces*

$$U = EAP,$$

*then for every  $b \in \mathbb{F}_p^m$ , the original system  $Ax = b$  is equivalent to*

$$Uy = Eb, \quad x = Py.$$

*Proof.* All elementary row operations used over  $\mathbb{F}_p$  are invertible, and column permutations are invertible. Therefore the same equivalence chain as in the binary case applies.  $\square$

The implementation advantage is weaker than over  $\mathbb{F}_2$ : additions are no longer xor-only, pivot normalization requires modular inversion, and dense bit-packing is less direct. For small fields of even characteristic, specialized libraries such as M4RIE extend related ideas beyond  $\mathbb{F}_2$  [13].

## 7 Iterative Lifting modulo $2^k$

Consider the system

$$Ax \equiv b \pmod{2^k}.$$

The reusable  $\mathbb{F}_2$  solver appears naturally when constructing a solution layer by layer.

**Theorem 6** (One-step lifting criterion). *Let  $A \in \mathbb{Z}^{m \times n}$  and  $b \in \mathbb{Z}^m$ . Suppose  $x_t$  satisfies*

$$Ax_t \equiv b \pmod{2^t}.$$

*Define*

$$r_t = b - Ax_t.$$

*Since  $x_t$  is a solution modulo  $2^t$ , every entry of  $r_t$  is divisible by  $2^t$ . Let*

$$c_t = \left( \frac{r_t}{2^t} \right) \bmod 2 \in \mathbb{F}_2^m.$$

*Then a vector*

$$x_{t+1} = x_t + 2^t y_t$$

*satisfies*

$$Ax_{t+1} \equiv b \pmod{2^{t+1}}$$

*if and only if*

$$(A \bmod 2)y_t = c_t$$

*over  $\mathbb{F}_2$ .*

*Proof.* The desired condition is

$$A(x_t + 2^t y_t) \equiv b \pmod{2^{t+1}}.$$

Equivalently,

$$2^t Ay_t \equiv b - Ax_t \pmod{2^{t+1}}.$$

By the definition of  $c_t$ ,

$$b - Ax_t \equiv 2^t c_t \pmod{2^{t+1}}.$$

Thus

$$2^t Ay_t \equiv 2^t c_t \pmod{2^{t+1}}.$$

Dividing by  $2^t$  gives the congruence modulo 2:

$$(A \bmod 2)y_t = c_t.$$

□



---

**Algorithm 4** Lifting modulo  $2^k$  using a reusable  $\mathbb{F}_2$  solver

---

**Require:**  $A \in \mathbb{Z}^{m \times n}$ ,  $b \in \mathbb{Z}^m$ , integer  $k \geq 1$

**Ensure:** Either UNSOLVABLE or one solution modulo  $2^k$

```

1: Preprocess  $A \bmod 2$  over  $\mathbb{F}_2$ 
2:  $x \leftarrow 0$ 
3: for  $t = 0, 1, \dots, k - 1$  do
4:   Compute  $r_t = b - Ax \pmod{2^{t+1}}$ 
5:    $c_t \leftarrow (r_t / 2^t) \bmod 2$ 
6:   if  $(A \bmod 2)y = c_t$  is not solvable over  $\mathbb{F}_2$  then
7:     return UNSOLVABLE
8:   end if
9:   Let  $y_t$  be one solution returned by the reusable  $\mathbb{F}_2$  solver
10:   $x \leftarrow x + 2^t y_t \pmod{2^{t+1}}$ 
11: end for
12: return  $x \pmod{2^k}$ 

```

---

**Remark 4** (Rank-deficient cases). *The lifting algorithm is sound: every returned vector satisfies the target congruence. However, if  $A \bmod 2$  is rank-deficient, choosing one arbitrary correction at each layer may lose completeness, because a different choice in the affine correction space may be needed to lift further. Complete treatment of rank-deficient cases requires tracking affine solution spaces across layers or using canonical forms over residue rings, such as Smith normal form or related methods[8, 9].*

## 8 CRT Framework for Composite Moduli

Let

$$m = \prod_{i=1}^s q_i, \quad q_i = p_i^{e_i},$$

where the  $q_i$  are pairwise coprime prime powers.

**Theorem 7** (CRT decomposition). *The system*

$$Ax \equiv b \pmod{m}$$

*is solvable if and only if every local system*

$$Ax_i \equiv b \pmod{q_i}$$

*is solvable. If  $x_i$  is a solution modulo  $q_i$  for every  $i$ , then there exists a unique  $x \pmod{m}$  such that*

$$x \equiv x_i \pmod{q_i}$$

*for all  $i$ , and this  $x$  solves the original system modulo  $m$ .*

*Proof.* The Chinese Remainder Theorem gives the ring isomorphism

$$\mathbb{Z}/m\mathbb{Z} \cong \prod_{i=1}^s \mathbb{Z}/q_i\mathbb{Z}.$$

Applying this isomorphism coordinatewise to vectors and matrices preserves addition and multiplication. Therefore the global equation is equivalent to the collection of local equations. The coordinatewise CRT reconstruction gives the unique global solution modulo  $m$ . Standard references include[6, 7, 5].  $\square$

CRT is an outer decomposition-and-reconstruction layer, not a local linear-algebraic optimization. Once the modulus is decomposed into prime powers, each local system can be handled by an appropriate prime-power solver. For the factor  $2^k$ , the lifting framework above directly applies.

## 9 Experimental Methodology

A complete experimental section should compare at least three implementations:

- (i) repeated bit-packed elimination for every RHS;
- (ii) reusable bit-packed preprocessing with online RHS queries;
- (iii) reusable preprocessing with batch RHS processing, and optionally M4RI-style preprocessing.

For reproducibility, the final version should report:

- (i) hardware model and CPU frequency;
- (ii) compiler and optimization flags;
- (iii) machine word size  $w$  and RHS packing width  $w_R$ ;
- (iv) matrix sizes  $m, n$ , number of right-hand sides  $Q$ , and density;
- (v) whether matrices are forced to be full rank;
- (vi) random seed policy;
- (vii) number of repetitions and whether times are mean, median, or minimum;
- (viii) raw running times, not only speedups.

| Size $n$ | Repeated elimination | Reusable online | Speedup      |
|----------|----------------------|-----------------|--------------|
| 512      | to be filled         | to be filled    | $2.93\times$ |
| 1024     | to be filled         | to be filled    | $3.82\times$ |
| 2048     | to be filled         | to be filled    | $5.15\times$ |
| 4096     | to be filled         | to be filled    | $> 11\times$ |

Table 1: Example benchmark table template. The final paper should include raw running times and the exact experimental setup.

The expected growth models are

$$T_{\text{repeat}}(n, Q) = \Theta\left(Q \frac{n^3}{w}\right),$$

$$T_{\text{online}}(n, Q) = \Theta\left(\frac{n^3}{w} + Qn^2\right),$$

and, for packed batch RHS,

$$T_{\text{batch}}(n, Q) = \Theta\left(\frac{n^3}{w} + n^2 \left\lceil \frac{Q}{w_R} \right\rceil\right),$$

with possible logarithmic improvements in preprocessing when M4RI-style elimination is used.

## 10 Conclusion

We presented a formal preprocessing-query framework for fixed-matrix many-RHS linear solving over  $\mathbb{F}_2$ . The basic solver records row operations, column permutations, pivot information, and an echelon form so that new right-hand sides can be solved without re-eliminating the fixed matrix. In the dense square bit-packed model, this changes the total cost from  $\Theta(Qn^3/w)$  to  $\Theta(n^3/w + Qn^2)$  for online queries.

We then strengthened the framework with two engineering directions. Batch RHS processing stores all right-hand sides as a matrix and applies row operations to many RHS columns per machine word. M4RI-style block elimination accelerates dense preprocessing by using lookup tables of pivot-row combinations, giving an additional logarithmic improvement in the dense model up to implementation-dependent overheads.

Finally, we connected the reusable  $\mathbb{F}_2$  solver to iterative lifting modulo  $2^k$  and CRT decomposition for composite moduli. Future work includes a full M4RI-backed implementation, SIMD and cache-blocking optimizations, systematic benchmarking against existing dense  $\mathbb{F}_2$  libraries, and complete treatment of rank-deficient lifting over prime-power residue rings.

## References

- [1] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, fourth edition, 2013.
- [2] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. SIAM, third edition, 1999.
- [3] Rudolf Lidl and Harald Niederreiter. *Finite Fields*. Cambridge University Press, second edition, 1997.
- [4] Joachim von zur Gathen and Jürgen Gerhard. *Modern Computer Algebra*. Cambridge University Press, third edition, 2013.
- [5] Henri Cohen. *A Course in Computational Algebraic Number Theory*. Springer, 1993.
- [6] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, sixth edition, 2008.
- [7] Kenneth Ireland and Michael Rosen. *A Classical Introduction to Modern Number Theory*. Springer, second edition, 1990.
- [8] Morris Newman. *Integral Matrices*. Academic Press, 1972.
- [9] Arne Storjohann. *Algorithms for Matrix Canonical Forms*. PhD thesis, ETH Zurich, 2000.
- [10] Gregory V. Bard. *Algorithms for Solving Linear and Polynomial Systems of Equations over Finite Fields*. PhD thesis, University of Maryland, 2007.
- [11] Martin R. Albrecht, Gregory V. Bard, and William Hart. Algorithm 898: Efficient multiplication of dense matrices over  $\mathbb{F}_2$ . *ACM Transactions on Mathematical Software*, 37(1), Article 9, 2010.
- [12] Martin R. Albrecht, Gregory V. Bard, and William Hart. Efficient multiplication of dense matrices over  $\mathbb{F}_2$ . arXiv:0811.1714, 2008.
- [13] Martin R. Albrecht. The M4RIE library for dense linear algebra over small fields with even characteristic. arXiv:1111.6900, 2011.

- [14] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13:354–356, 1969.
- [15] Victor Shoup. NTL: A Library for doing Number Theory. <https://libntl.org/>.